

 Universidad Católica de Cuenca	LISTA DE COTEJO PARA LA EVALUACIÓN DEL PROYECTO RETO
---	---

**UNIDAD ACADÉMICA DE INFORMÁTICA, CIENCIAS DE LA
COMPUTACIÓN, E INNOVACIÓN TECNOLÓGICA**
CARRERA DE SISTEMAS BIOMÉDICOS
Período Lectivo MARZO 2026 – AGOSTO 2026

Asignatura:	Infraestructura de Sistemas Clínicos en la Nube – Gestión de Proyectos Tecnológicos Sanitarios	Fecha:	20 de julio del 2026
Docente:	Ing. Fanny Chuchuca, MSc. Ing. Pablo Salamea, Msc.		
Ciclo:	7mo	Bloque Temático N°:	3
TEMA:	Diseño e Implementación de un Repositorio Clínico Único en la Nube para una Red Hospitalaria Oncológica Multi-Sede		
Nombre del estudiante:			

AVANCE 1 — Diseño inicial y microservicios base
Repositorio Clínico Único en la Nube para una Red Hospitalaria Oncológica Multi-Sede

1. Objetivo del avance

El primer avance tuvo como objetivo plantear la arquitectura inicial del repositorio medico regional de SOLCA y construir los microservicios base de forma independiente. La necesidad principal era resolver la fragmentacion de informacion clinica entre SOLCA Cuenca, SOLCA Manabi y SOLCA Quito, donde cada sede puede tener datos separados de pacientes, consultas, laboratorio e imagenologia.

2. Analisis del problema actual

El problema identificado es que la informacion oncologica se encuentra distribuida entre sedes y dominios clinicos. Un paciente puede tener datos administrativos en una sede, consultas en otra, resultados de laboratorio en otra fuente y estudios de imagenologia en otro sistema. Esto dificulta la continuidad asistencial, genera duplicidad de pacientes y obliga al personal medico a revisar varias fuentes para reconstruir la historia regional.

Para responder a ese problema se diseno un repositorio regional basado en microservicios. Cada microservicio mantiene su propio dominio y su propia base de datos, mientras el Repositorio Clinico Regional funciona como punto de consulta central. En esta primera fase el repositorio podia existir sin integracion completa, dejando preparada la base para el Avance 2.

3. Que hicimos

Se crearon microservicios Spring Boot separados para los dominios solicitados:

Microservicio	Carpeta	Puerto	Base de datos
Paciente Maestro Regional	paciente-maestro-service	8081	paciente_maestro_db
Consulta Clinica	consulta-clinica-service	8082	consulta_clinica_db
Laboratorio Clinico	laboratorio-clinico-service	8083	laboratorio_clinico_db
Imagenologia / PACS basico	imagenologia-service	8084	imagenologia_db
Repositorio Clinico Regional	repositorio-clinico-regional-service	8085	repositorio_clinico_regional_db

Tambien se agrego una estructura de base de datos independiente mediante database/init/01-create-databases.sql, donde cada dominio clinico tiene una base PostgreSQL separada.

4. Como lo hicimos

Cada microservicio se implemento con:

- Java 17 y Spring Boot.
- API REST basica por dominio.
- Spring Data JPA para persistencia.
- PostgreSQL como motor de base de datos.
- application.properties independiente por servicio.
- Datos semilla en data.sql para probar rapidamente los endpoints.
- Dockerfile individual para permitir construccion aislada.

La organizacion del codigo permite ejecutar cada servicio de manera separada o levantar todo el entorno con docker-compose.yml.

5. Diseno preliminar de arquitectura cloud

La arquitectura preliminar se documento con diagramas generados en la carpeta documentacion/assets/:

- diagrama_arquitectura_cloud_solca.png
- diagrama_componentes_cloud_solca.png
- diagrama_modelo_datos_solca.png

El diseno considera una aplicacion frontend, microservicios backend, bases de datos independientes y un repositorio regional como agregador. En esta fase se definio la separacion de responsabilidades, aunque la integracion completa se trabajo en el Avance 2.

6. Modelo del Paciente Maestro Regional

El Paciente Maestro Regional se definio como la identidad minima comun para relacionar los registros clinicos entre sedes. El modelo se implemento en:

paciente-maestro-service/src/main/java/ec/edu/solca/paciente/model/Paciente.java

Campos principales:

- idPacienteRegional
- cedula
- nombres
- apellidos
- sedeOrigen
- fechaNacimiento
- sexo

- direccion
- telefono

Este identificador regional permite que consultas, laboratorio e imagenologia se vinculen al mismo paciente sin depender de una sola base centralizada.

7. Diseno de bases de datos independientes

La independencia de datos se cumple con bases separadas:

```
CREATE DATABASE paciente_maestro_db;
CREATE DATABASE consulta_clinica_db;
CREATE DATABASE laboratorio_clinico_db;
CREATE DATABASE imagenologia_db;
CREATE DATABASE repositorio_clinico_regional_db;
```

Cada microservicio se conecta solo a su base mediante su propia variable `SPRING_DATASOURCE_URL`. Esto cumple el principio de base de datos por microservicio.

8. Componentes cloud identificados

Componente	Uso planteado en el proyecto
IaaS	VPS o maquinas virtuales para ejecutar contenedores cuando se requiere control de infraestructura.
PaaS	Plataforma para desplegar microservicios Spring Boot y frontend sin administrar servidores manualmente.
SaaS	GitHub para repositorios, Postman para pruebas y herramientas de colaboracion.
DBaaS	PostgreSQL administrado para alojar las bases independientes de cada microservicio.

9. Endpoints minimos funcionales

Servicio	Endpoints principales
Paciente Maestro	GET /pacientes, GET /pacientes/{id}, GET /pacientes/cedula/{cedula}, POST /pacientes
Consulta Clinica	GET /consultas, GET /consultas/paciente/{idPacienteRegional}, POST /consultas
Laboratorio Clinico	GET /laboratorio, GET /laboratorio/paciente/{idPacienteRegional}, POST /laboratorio
Imagenologia	GET /imagenes, GET /imagenes/paciente/{idPacienteRegional}, POST /imagenes
Repositorio Regional	Servicio creado para preparar la integracion posterior

10. Evidencias disponibles

Evidencia solicitada	Donde esta en el proyecto
Repositorio GitHub creado	Backend y frontend publicados en GitHub.
Codigo organizado por microservicios	Carpetas paciente-maestro-service, consulta-clinica-service, laboratorio-clinico-service, imagenologia-service, repositorio-clinico-regional-service.
Pruebas Postman	postman/SOLCA-Repositorio-Clinico-Regional.postman_collection.json.
Capturas de bases de datos	Se pueden tomar desde PostgreSQL/psql usando las bases creadas en database/init/01-create-databases.sql.

Evidencia solicitada	Donde esta en el proyecto
Diagrama preliminar	documentacion/assets/diagrama_arquitectura_cloud_solca.png y documentacion/assets/diagrama_modelo_datos_solca.png.

11. Cumplimiento de la rubrica

Requisito	Estado	Explicacion
Analisis del problema SOLCA Cuenca, Manabi y Quito	Cumple	Se documento la fragmentacion de informacion entre sedes y la necesidad de identidad regional.
Diagrama preliminar de arquitectura cloud	Cumple	Existe diagrama de arquitectura cloud en documentacion/assets.
Modelo del Paciente Maestro Regional	Cumple	Implementado como entidad Paciente y documentado como identidad regional.
Bases de datos independientes	Cumple	Cada microservicio usa una base PostgreSQL propia.
Componentes IaaS, PaaS, SaaS, DBaaS	Cumple	Identificados en la documentacion y diagramas cloud.
Microservicios creados y funcionando	Cumple	Existen cinco microservicios base con API REST y persistencia.
API REST basica	Cumple	Cada servicio expone endpoints GET/POST minimos.
Pruebas en Postman	Cumple parcialmente	Existe coleccion Postman; las capturas deben tomarse al ejecutar las pruebas.
Capturas de bases de datos	Pendiente de evidencia visual	Las bases existen; faltan capturas guardadas como imagen.

12. Conclusion

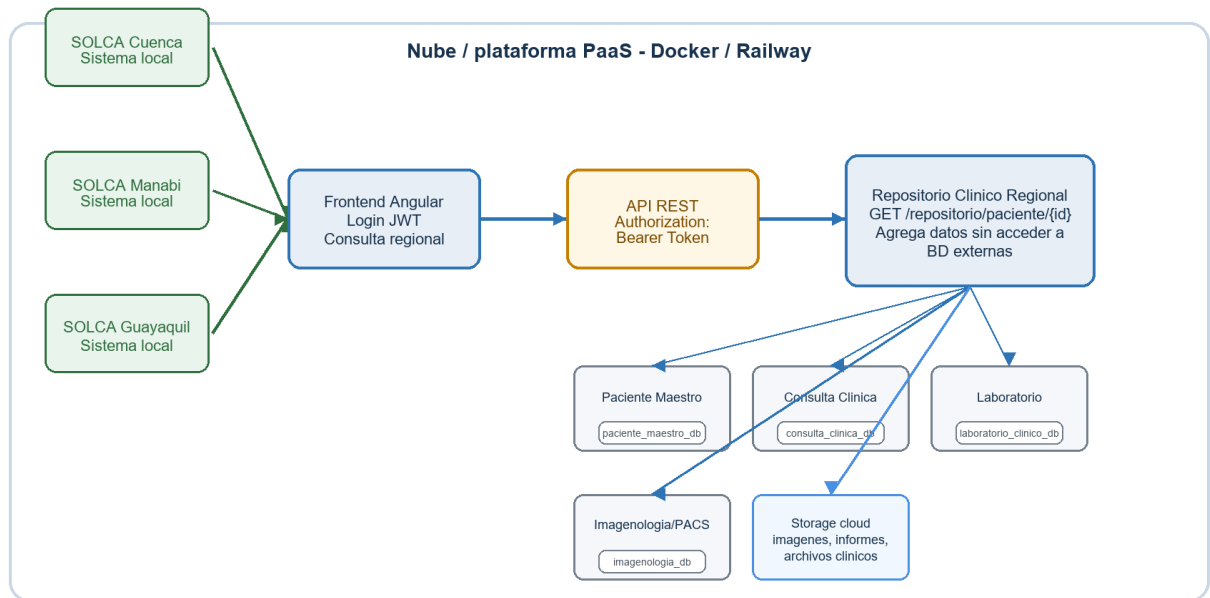
El Avance 1 queda cubierto porque el proyecto define la arquitectura inicial, separa los dominios clinicos en microservicios independientes, crea bases PostgreSQL separadas y deja disponible una coleccion Postman para pruebas. Las unicas evidencias que deben anexarse manualmente son capturas visuales de Postman y de las bases de datos durante la ejecucion.

Anexos: diagramas

Como evidencia del diseño e implementación se incluyen los diagramas generados para el proyecto.

Diagrama preliminar de arquitectura cloud

Arquitectura cloud preliminar - Repositorio Clinico Regional SOLCA

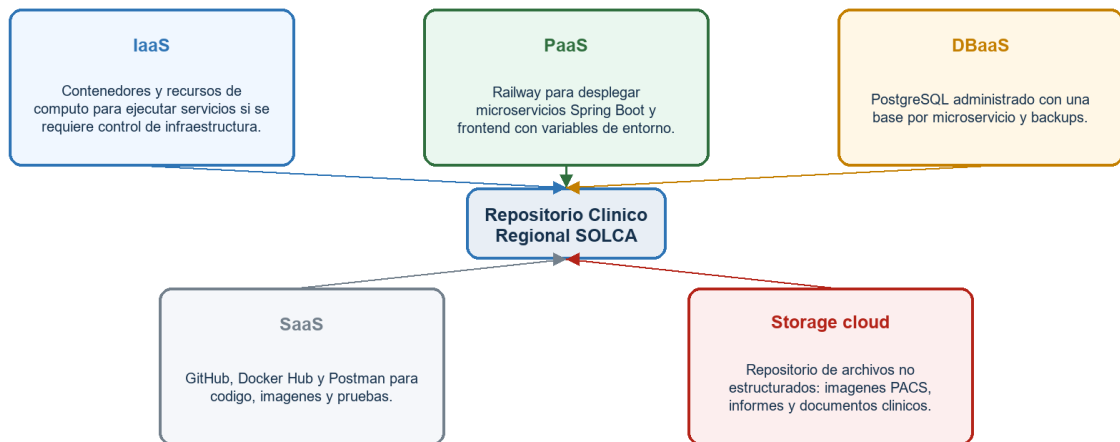


Principio clave: cada microservicio conserva su propia base de datos; la integración se hace por APIs REST.

Diagrama preliminar de arquitectura cloud

Componentes cloud identificados

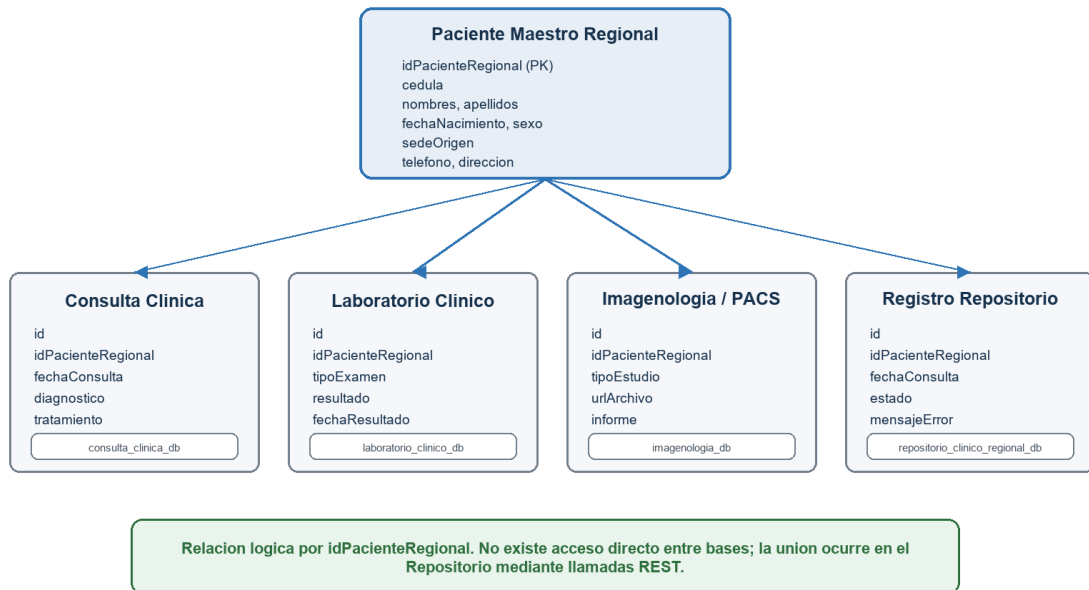
Componentes cloud identificados para el proyecto



Componentes cloud identificados

Modelo de datos del Paciente Maestro Regional

Modelo de datos: Paciente Maestro y bases independientes



Modelo de datos del Paciente Maestro Regional